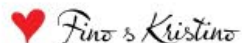


Christian Nagel

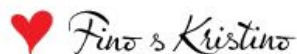
Microsoft MVP

How Aspire helps developing all .NET solutions (and more)



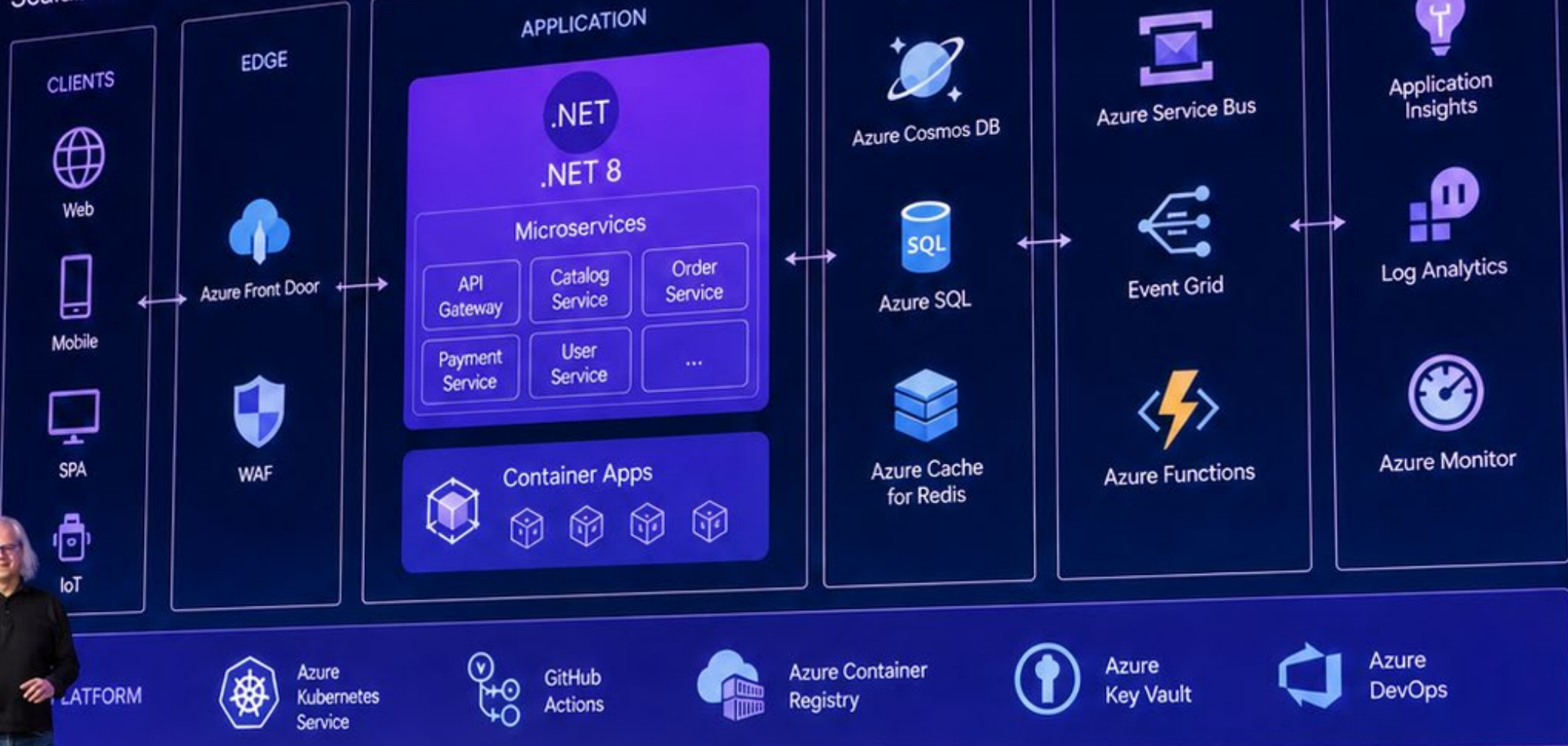


We would like to thank our sponsors



Cloud-native .NET

Scalable. Resilient. Observable.



.NET
Conf 2024

.NET

Christian Nagel

- 20+ years of experience with C# and .NET
- Author of 20+ books on C# and .NET (latest: "Modern C# in .NET 10")
- Microsoft MVP for C# and .NET
- Speaker at global conferences (TechEd, BASTA!, Thrive... for 20+ years)
- Passionate about language design and developer productivity



Topics

1. Aspire - what's this?
2. Aspirefy your codebase - how to adopt new features incrementally
3. Aspire integrations
4. Aspire with desktop apps
5. Aspire with web apps

1) Aspire – What's this?

*Aspire helps teams **design, run, observe, deploy, and evolve** distributed solutions with fewer manual steps.*

Aspire Intro

Orchestrate frontends, APIs, containers, and databases effortlessly—no rewrites, no limits. Extend Aspire to power any project. Free, open-source, and agent ready.

- Local-first, production ready - develop and test distributed apps on your machine with the same experience as in production
- OpenTelemetry dashboard - built-in observability for all your services
- Integrations - make it easier to connect to databases, messaging, and more
- Multi-language support - Use Aspire with .NET, Java, Python, and more

1) Aspire - what's this?

Orchestrate frontends, APIs, containers, and databases effortlessly—no rewrites, no limits. Extend Aspire to power any project. Free, open-source, and agent ready.

- Local-first, production ready - develop and test distributed apps on your machine with the same experience as in production
- OpenTelemetry dashboard - built-in observability for all your services
- Integrations - make it easier to connect to databases, messaging, and more
- Multi-language support - Use Aspire with .NET, Java, Python, and more

2) Aspireify your codebase

Adoption is incremental, not all-or-nothing.

- Start with one service
- Introduce service defaults
- Add telemetry and health signals
- Expand to other components step by step



Practical Aspirefy roadmap

Step 1

- Create AppHost
- Model current dependencies

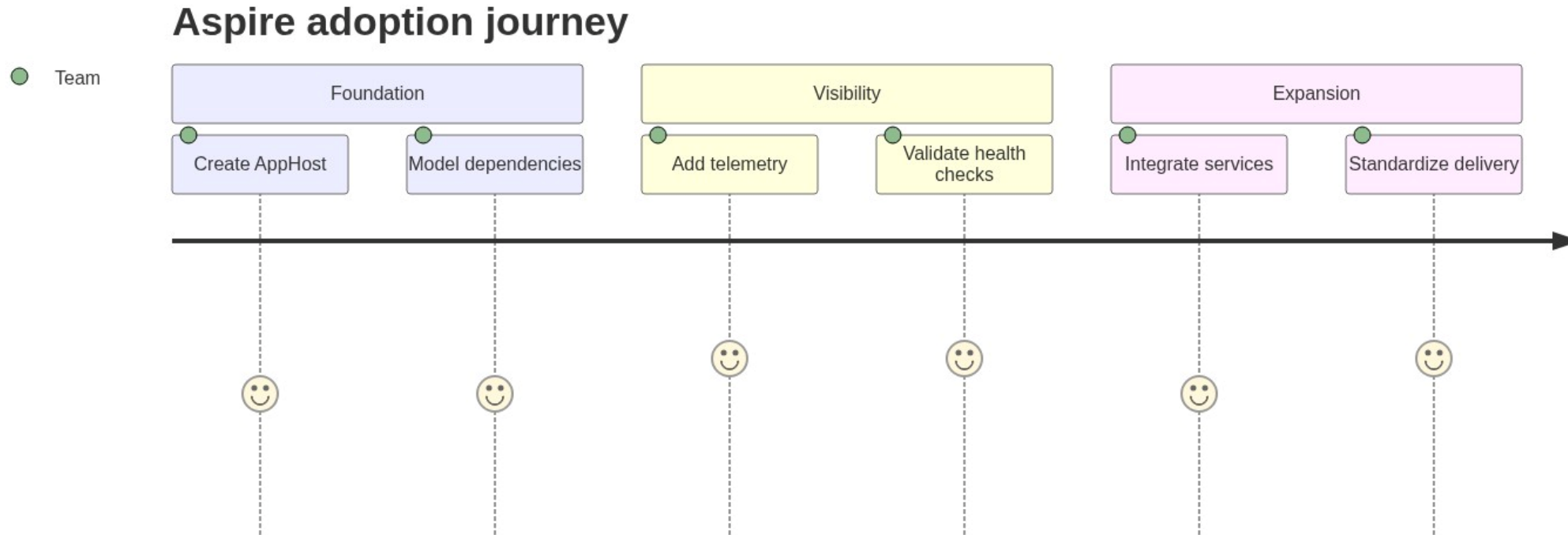
Step 2

- Introduce observability baseline
- Validate local developer workflow

Step 3+

- Add integrations intentionally
- Move service by service

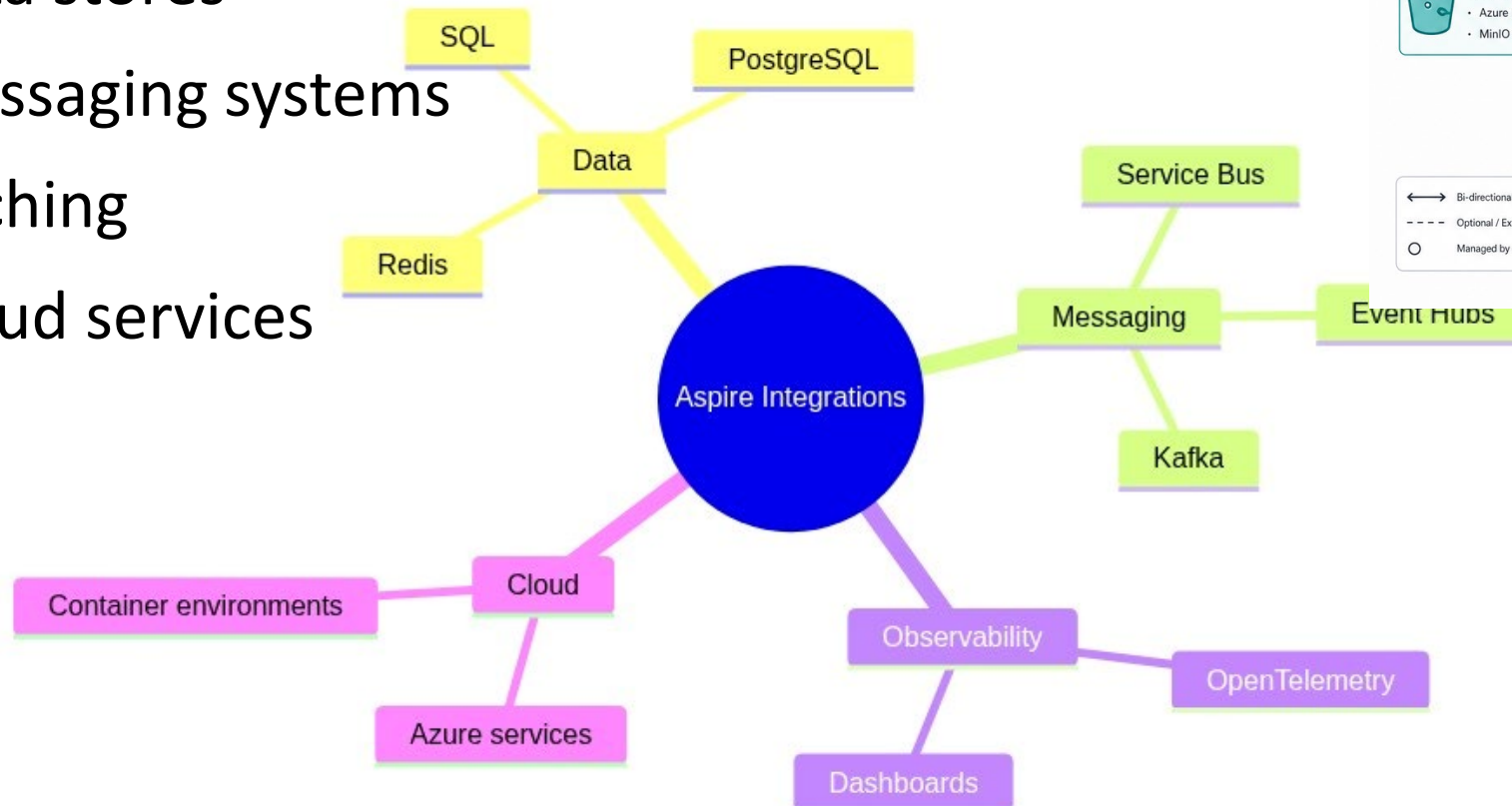
Practical Aspireify roadmap



3) Aspire integrations

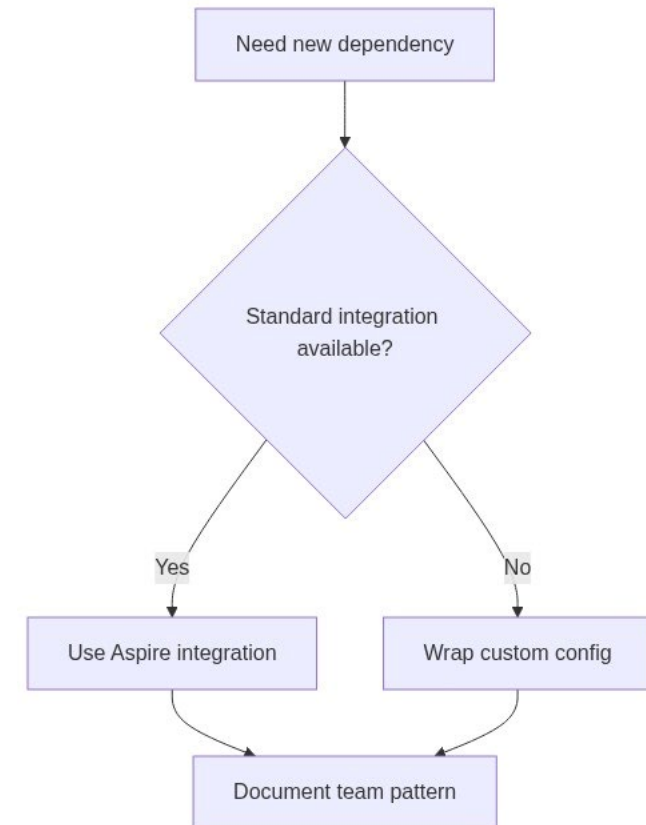
Integrations reduce boilerplate and configuration drift.

- Data stores
- Messaging systems
- Caching
- Cloud services



Integration decision checklist

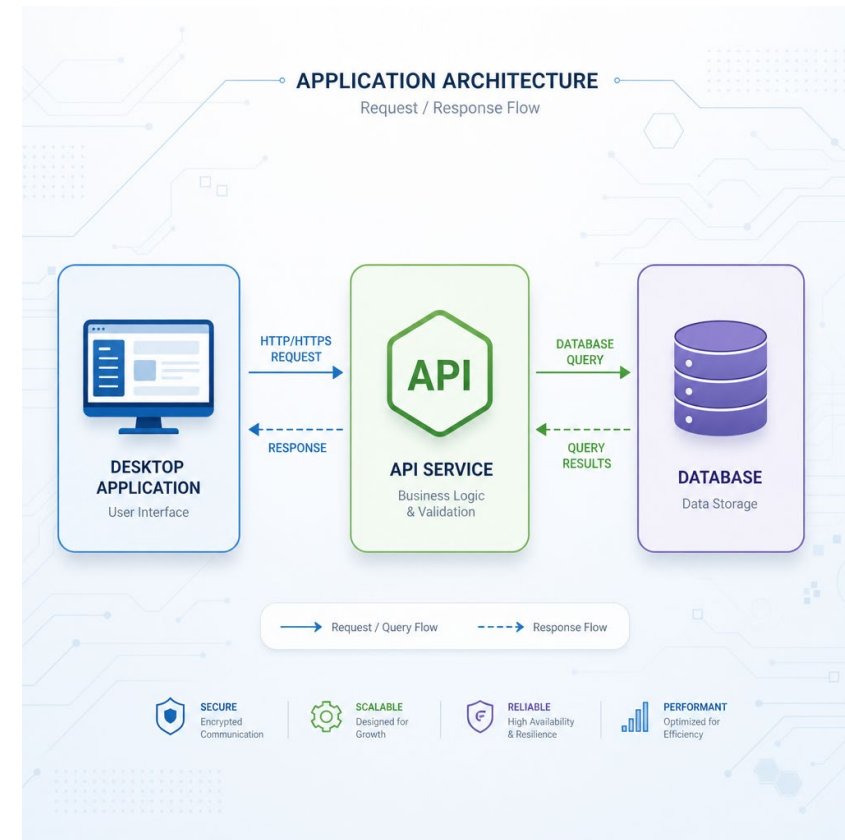
- Does it improve local + cloud parity?
- Does it reduce custom infrastructure code?
- Does it improve reliability and diagnostics?
- Can the team support it operationally?



4) Aspire with desktop apps

Desktop apps can benefit from Aspire as the orchestration and diagnostics backbone.

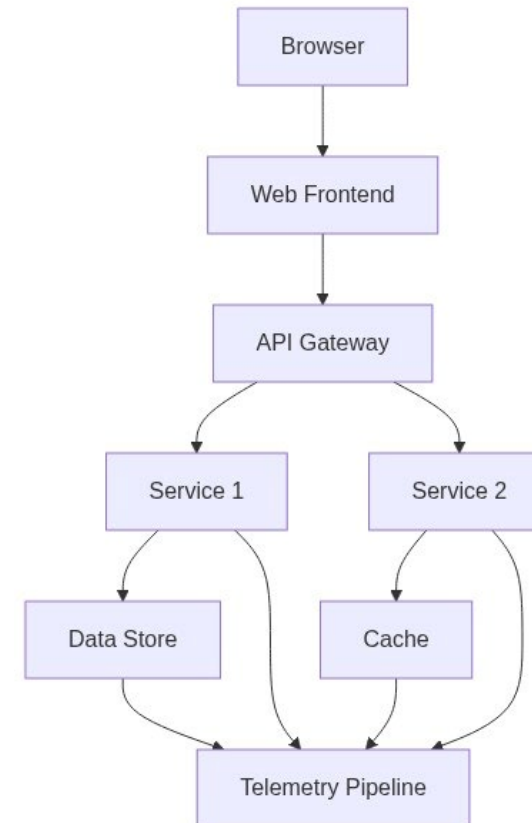
- Keep rich client UX
- Orchestrate backend services consistently
- Observe end-to-end interactions



5) Aspire with web apps

Web architectures gain standardization and visibility fast.

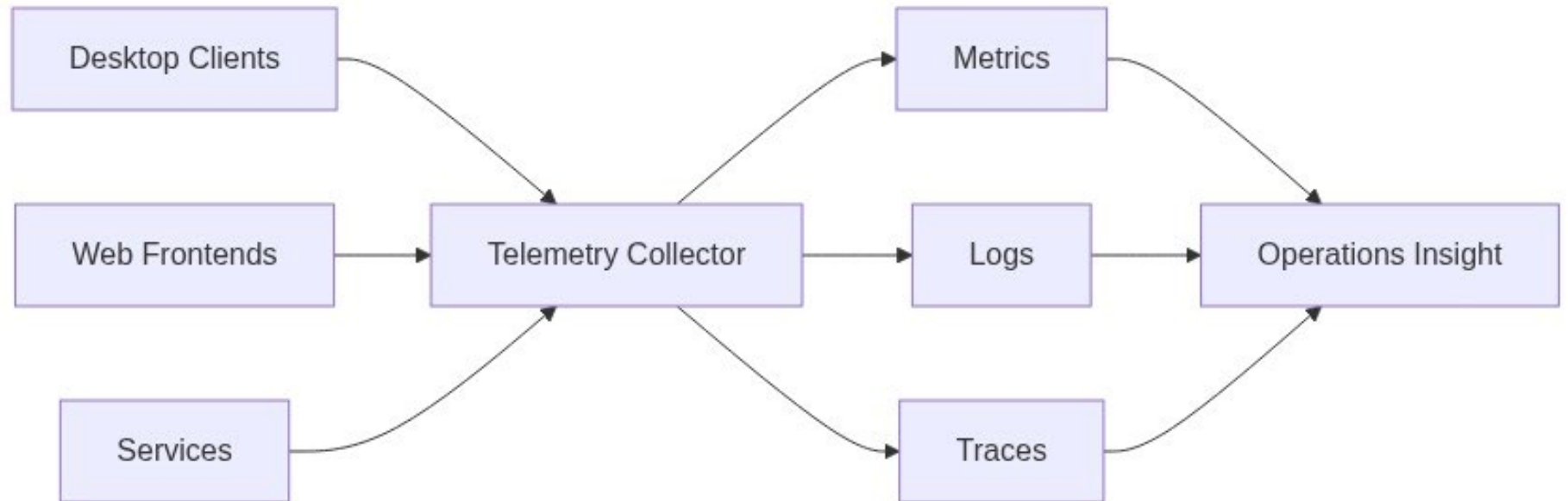
- Cleaner service composition
- More predictable environments
- Faster issue diagnosis



Cross-cutting observability (desktop + web)

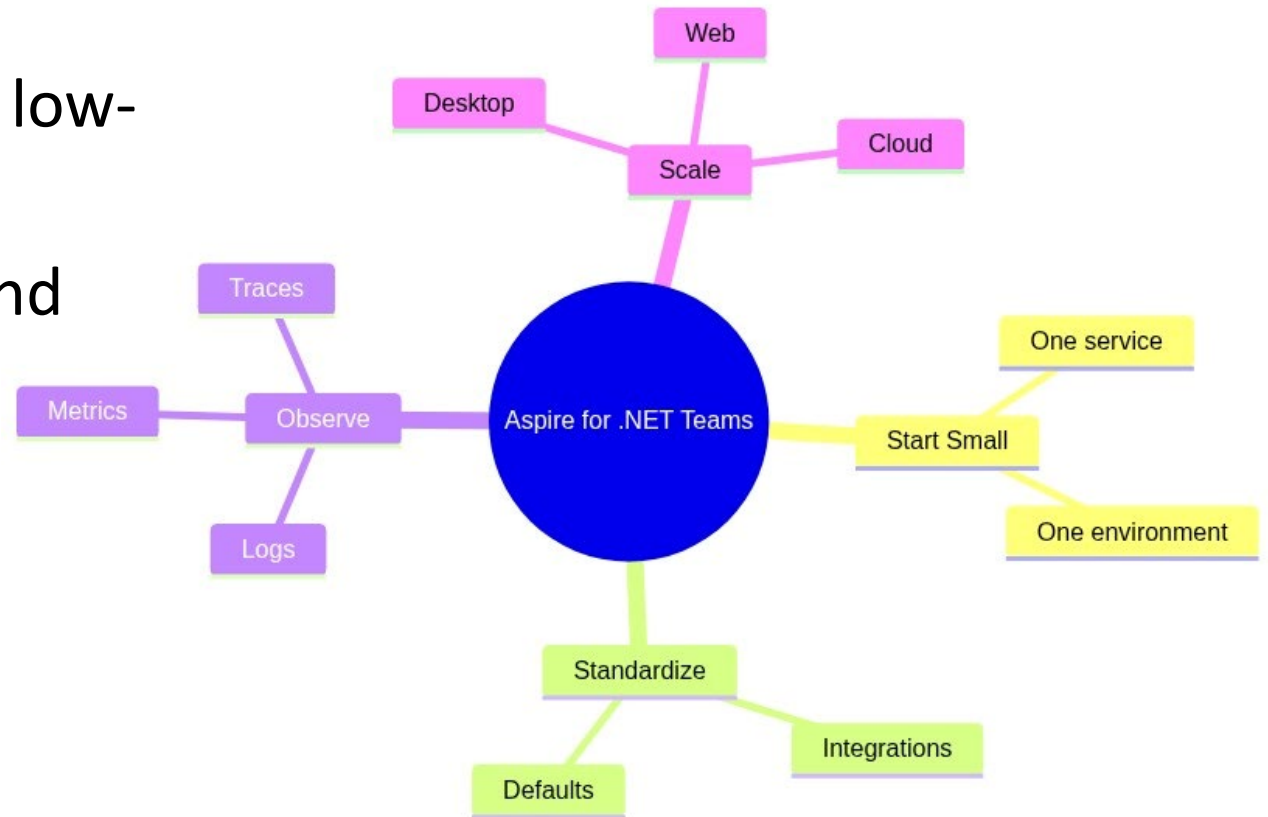
One operational language across all app types:

- Metrics for trends
- Logs for context
- Traces for causality



Summary

- Aspire provides a practical path to distributed apps
- Adoption can be incremental and low-risk
- Integrations accelerate delivery and reduce friction
- Desktop and web scenarios both benefit from the same model



Resources

- [Aspire documentation - https://aspire.dev/](https://aspire.dev/)
- [Aspire samples - https://github.com/microsoft/aspire-samples](https://github.com/microsoft/aspire-samples)
- [OpenTelemetry for .NET - https://learn.microsoft.com/dotnet/core/diagnostics/observability-with-otel](https://learn.microsoft.com/dotnet/core/diagnostics/observability-with-otel)
- [Codebreaker Org - https://github.com/codebreakerapp](https://github.com/codebreakerapp)
- [Pragmatic Microservices with C# and Azure - https://www.cninnovation.com/books/](https://www.cninnovation.com/books/)
- [My blog - https://csharp.christiannagel.com/](https://csharp.christiannagel.com/)

Q & A

Common questions:

- *Do I need microservices to use Aspire?* — No, you can start with modular monolith + selected services.
- *Do I need Docker to use Aspire?* — No, Docker is only required depending on the integrations you use.
- *Is Aspire only for cloud?* — No, it improves local development and also allows deploying to on-premises environments.
- *Can desktop apps use Aspire?* — Yes, especially for distributed backends and observability.
- *Do we need to migrate everything at once?* — No, incremental adoption is the recommended path.

Thank You!

Enjoy Thrive Conference!

Build distributed .NET systems with clarity, consistency, and confidence.

Thrive Conference 2026

Slides and samples: github.com/CNILearn/thrive2026

Thank you

We greatly value your input. Please take a moment to complete our survey, accessible either through the Run.events app or by the link below:

<https://bit.ly/Thrive26Survey>

